Seguridad de archivos mapeados con mmap en moon

Sistemas Operativos | Parcial 3: Seguridad de Archivos en Capa Kernel y mmap

Pablo Manjarres

Objetivo y uso

El módulo `kernel/mmap_guard.ko` protege un archivo: niega mapearlo con escritura compartida o con ejecución, también cuando se intenta con `mprotect()`. El comando `m_fmap` de moon mapea un archivo y muestra cada llamada al sistema. Requiere Linux, gcc, make, sudo y el árbol de compilación del kernel en ejecución.

```
make && make -C kernel                # moon y mmap_guard.ko
sudo insmod kernel/mmap_guard.ko path=/tmp/moon_secret.txt
./moon
m_fmap /tmp/moon_secret.txt r         # lectura compartida: permitida
m_fmap /tmp/moon_secret.txt w         # escritura compartida: mmap -> -1 EACCES
sh tests/run_mmap_guard_tests.sh      # ataques, registro y contadores
sh tests/mmap_overhead.sh             # sobrecosto con ftrace y strace
sh kernel/demo.sh                     # demostración paso a paso
```

Los modos de `m_fmap` son: `r` lectura, `w` escritura compartida, `p` copia privada, `x` ejecución y `u` lectura ampliada con `mprotect()`.

Parte I: preguntas diagnósticas

1. De mmap() al primer fallo de página. `mmap()` entra a Ring 0 por `syscall`: `ksys_mmap_pgoff()` → `vm_mmap_pgoff()`. Allí `security_mmap_file()` consulta los LSM. Con `mmap_lock` tomado, `do_mmap()` calcula `vm_flags` y `mmap_region()` crea un `vm_area_struct` con `vm_file`, `vm_pgoff` y los `vm_ops` del sistema de archivos. No se crea ninguna PTE. En el primer acceso, la MMU recorre PGD→P4D→PUD→PMD→PTE, no encuentra una entrada presente y lanza `#PF` con la dirección en CR2. `exc_page_fault()` llama a `handle_mm_fault()`, que valida el acceso contra `vm_flags` y llama a `do_fault()`. En una lectura, `filemap_fault()` trae la página del page cache, o del disco, e instala una PTE de solo lectura. En una escritura privada, `do_cow_fault()` copia la página. En una escritura compartida, `do_shared_fault()` avisa al sistema de archivos con `->page_mkwrite`. Desde ahí, los accesos no pasan por el kernel. Por eso el control debe hacerse al crear la VMA.

2. MAP_SHARED frente a MAP_PRIVATE. `do_mmap()` traduce `MAP_SHARED` a `VM_SHARED | VM_MAYSHARE`: las PTE apuntan a la misma página del page cache que ven `read()` y los demás procesos. `MAP_PRIVATE` no lleva `VM_SHARED`: la primera escritura hace COW y crea una página anónima del proceso. Con un descriptor `O_RDONLY`, el kernel quita `VM_MAYWRITE | VM_SHARED` (`mm/mmap.c:464`), así que el mapeo nunca puede pasar a escritura. En seguridad, con `MAP_SHARED` una escritura persiste en disco y todos los procesos la ven de inmediato, así que un proceso comprometido corrompe los datos de todos. Con `MAP_PRIVATE`, los cambios no persisten ni se comparten. Pero la copia queda en memoria anónima del proceso, que puede ir a swap o a un volcado de memoria.

3. Copy-On-Write. La página empieza compartida con el page cache y con una PTE sin escritura, aunque la VMA tenga `VM_WRITE`. La primera escritura lanza `#PF`. `do_wp_page()` comprueba que la VMA permite escribir, copia la página a una anónima nueva, instala la PTE escribible y suelta la referencia a la original. El peligro aparece cuando el kernel concede escritura donde solo hay COW. Pasaría si `mprotect` aceptara `PROT_WRITE` sin `VM_MAYWRITE`, o si `get_user_pages()` con `FOLL_FORCE` escribiera en la página del page cache en vez de en la copia. Eso fue Dirty COW (CVE-2016-5195): se escribía en `/proc/self/mem` mientras otro hilo descartaba la copia con `MADV_DONTNEED`. Así, un usuario sin privilegios modificaba archivos de solo lectura, como `/etc/passwd` o binarios SUID, y obtenía root.

4. Tabla de syscalls frente a LSM. Un LKM con hook en `sys_call_table` se carga sin recompilar y ve cada syscall al entrar. Pero `kallsyms_lookup_name()` no está exportado desde 5.7, la tabla es de solo lectura y el bit CR0.WP está fijado desde 5.3. Además, no ve las rutas internas del kernel y lee argumentos crudos de usuario, que pueden cambiar después de revisarlos (TOCTOU). Su viabilidad en 6.x es muy baja.

Un LSM usa los ganchos `mmap_file` (llamado `file_mmap` antes de 3.5) y `file_mprotect`. Está en el camino de todas las rutas, recibe `struct file`, `prot` y `flags` ya validados, y es la API soportada. Su desventaja es que hay que compilarlo con el kernel, porque `security_add_hooks()` es `__init`. BPF LSM evita recompilar, pero requiere

`bpf` en `lsm=`. Elegí kretprobes sobre `security_mmap_file()` y `security_file_mprotect()`: ven lo mismo que un LSM y se cargan como módulo.

5. Cifrado transparente. El descifrado va debajo del page cache, en el sistema de archivos. fscrypt descifra en `->read_folio` y deja el texto claro en el page cache. Al escribir, cifra en una página temporal durante el writeback, sin tocar la página en caché. El acceso se controla al abrir el archivo: sin la clave en el keyring no se llega al contenido. Otro proceso no puede leer esa página, porque solo aparece en las tablas de páginas de quien la mapea, y `/proc/pid/mem` exige `ptrace_may_access()`. Al retirar la clave, fscrypt expulsa las páginas en claro. Contra la lectura física de la RAM o por DMA se usan cifrado de memoria (AMD SME/SEV, Intel TME) e IOMMU.

6. Páginas sucias, msync y fsync. En un mapeo compartido escribible, la PTE empieza sin permiso de escritura. La primera escritura a una página limpia produce un fallo: `do_shared_fault()` llama a `->page_mkwrite` y marca la página sucia (`folio_mark_dirty`). Después deja la PTE escribible, con el bit dirty que pone la MMU. El writeback escribe las páginas sucias con `->writepages`. Antes, con `folio_mkclean()`, quita otra vez la escritura de las PTE para detectar la próxima escritura. `msync(MS_SYNC)` y `fsync()` inician ese writeback y esperan a que termine. `munmap()` y la reclamación de memoria también pasan el bit dirty de la PTE a la página.

Así se evaden los filtros: un filtro que inspecciona `write()` no ve las escrituras por puntero. Además, `page_mkwrite` solo avisa en la primera escritura sobre una página limpia. Los bytes siguientes cambian sin aviso hasta el próximo writeback, así que un atacante puede escribir después de la inspección. Por eso el módulo impide crear el mapeo compartido escribible.

7. Auditoría con eBPF. Sin recompilar, bpftrace o libbpf cargan programas en kprobes y `fentry`/`fexit`. Se pueden poner sobre `security_mmap_file`, `security_file_mprotect` y `handle_mm_fault`, o en los tracepoints `syscalls:sys_enter_mmap` y `exceptions:page_fault_user`. El programa guarda pid, uid, inode, `prot` y dirección en mapas BPF o en un ring buffer. Con eso detecta patrones anómalos, como un proceso que mapea con escritura un inode sensible o un número inusual de fallos de escritura por segundo. El verificador garantiza que el programa no rompa el kernel. Para bloquear, BPF LSM (`lsm/mmap_file`, compilado en el kernel probado) devuelve `-EPERM`. Límite: después del primer fallo, los accesos por puntero no generan eventos.

8. O_RDONLY con PROT_WRITE y MAP_PRIVATE. Sí se permite. `do_mmap()` solo exige `FMODE_WRITE` para `MAP_SHARED` con `PROT_WRITE`. Con `MAP_PRIVATE`, la escritura cae en una copia COW anónima y nunca llega al archivo. Implica tres riesgos:

- **Alteración temporal.** El proceso cambia su propia vista hasta `munmap`, por ejemplo una configuración o un código ya cargados.
- **Fuga de información.** Los datos quedan en páginas anónimas que pueden ir a swap o a un volcado de memoria.
- **Cambios ajenos.** Las páginas aún no escritas siguen mostrando lo que otros escriban en el archivo.

Si COW falla, como en Dirty COW, esto se vuelve una escritura real. El módulo permite este caso y la prueba confirma que el archivo no cambia.

9. Carreras con MADV_DONTNEED. El parche de Dirty COW (commit 19be0eaffa3a, Linux 4.9) dejó de quitar `FOLL_WRITE` al reintentar y agregó `FOLL_COW`. En 6.0, `FOLL_COW` se eliminó (commit 5535be309971). Desde entonces, `can_follow_write_pte()` (`mm/gup.c:785`) solo permite escribir con `FOLL_FORCE` si la página es anónima y exclusiva de esa VMA (`PageAnonExclusive`), nunca en una del page cache. `madvise(MADV_DONTNEED)` y GUP pueden correr en paralelo con `mmap_lock` en lectura. El lock de la tabla de páginas serializa cada cambio de PTE, y si la PTE desaparece, GUP vuelve a resolver el fallo con una copia nueva. Desde 6.12, `proc_mem.force_override` (`fs/proc/base.c:126`) permite prohibir del todo esas escrituras forzadas.

10. struct page, tablas de páginas y bits de la MMU. Cada marco físico tiene un `struct page` o folio con un contador de referencias, `mapping` (page cache o anónima) y banderas. Con ellos, el kernel decide si una escritura va a la página compartida o a una copia. La MMU traduce cada dirección con PGD→P4D→PUD→PMD→PTE y verifica en hardware los bits de cada nivel. R/W impide escribir páginas de solo lectura, U/S impide que Ring 3 acceda a las páginas del kernel y NX impide ejecutar páginas de datos. SMEP y SMAP impiden que el kernel ejecute o lea memoria de usuario por error. Un archivo mapeado sin `PROT_EXEC` tiene NX en sus PTE, así que el código inyectado ahí produce `#PF`. Para que ese control no se pueda desactivar, el módulo niega `PROT_EXEC` y su ampliación con `mprotect()`.

Parte II: estrategia con IA

Flujo de trabajo

- **Exploración.** La IA comparó los puntos de intercepción: tabla de syscalls, LSM, kprobes, eBPF y `vm_ops->fault`. También leyó las cabeceras y propuso los ganchos `security_mmap_file()` y `security_file_mprotect()`. Antes de usar cada firma o símbolo, lo busqué en el árbol de Linux 6.18.
- **Código C.** La IA escribió el primer borrador del módulo, de `m_fmap` y de las pruebas. Para verificarlo, compilé con Kbuild y `-Wall -Wextra -Werror`, cargué el módulo y revisé `dmesg`.
- **Pruebas y PoC.** La IA propuso los vectores: escritura compartida, `MAP_SHARED_VALIDATE`, ampliación con `mprotect`, ejecución y la carrera Dirty COW. Cada prueba compara el archivo con `read()`, porque un código de retorno no basta.

Prompts estructurados

Contexto	Rol	Restricción	Resultado esperado
Enunciado del parcial; shell moon en C con dos parciales previos	Arquitecto del kernel Linux	Capa de kernel, cargable como módulo, sin parchear el kernel, compatible con 5.x y 6.x	Comparación de puntos de intercepción con riesgos y una recomendación
Kernel 6.18 de WSL2 con `CONFIG_KRETPROBES=y` y `CONFIG_MODVERSIONS=y`	Programador de módulos del kernel	Solo símbolos sondeables o exportados, sin `sys_call_table` ni CR0, la política en una función y cero advertencias	`mmap_guard.c`: niega escritura compartida y `PROT_EXEC` sobre un inode, con registro y contadores
Contrato `ui_trace_*` de moon y plantilla de `cat_data.c`	Desarrollador del shell	Mismo estilo que los comandos existentes; compila en Linux y macOS	Comando `m_fmap` que traza `open`, `mmap`, `mprotect`, `msync` y `munmap`
Política del módulo y CVE-2016-5195	Analista de seguridad ofensiva	Usuario sin privilegios con permiso POSIX de escritura; verificar con `read()`; grupo de control sin módulo	`mmap_attacks.c`: un PoC por vector y la carrera Dirty COW
Módulo cargado en WSL2, sin `perf`	Revisor de rendimiento	Usar ftrace y strace, con y sin módulo, en varias corridas	Costo por `mmap()` y por fallo de página

Errores de la IA detectados y corregidos

Afirmación o código de la IA	Problema	Detección y corrección
"La regla actual de `can_follow_write_pte()` existe desde 5.8"; después, "desde 6.1"	Las dos versiones son falsas: ese cambio (commit 5535be309971, que eliminó `FOLL_COW`) está en v6.0 y no en v5.19	Comparación de etiquetas en el repositorio de Linux; se corrigió a 6.0
"`proc_mem.force_override` existe desde 6.2"	Llegó en 6.12: el commit 41e8149c8892 no está en v6.11	Misma comparación; se corrigió
"Desactivar BTF y `DEBUG_INFO` para compilar el árbol más rápido no cambia la ABI de módulos"	`CONFIG_DEBUG_INFO_BTF_MODULES` agrega campos a `struct module` (`include/linux/module.h:510`), y el compilador era otro (gcc 15 en vez de 13)	`insmod`: "Invalid module format". Se recompiló con el `/proc/config.gz` exacto y gcc 13
Compilar contra `/lib/modules/$(uname -r)/build`	En WSL ese directorio solo trae `include/`, sin Makefile	`make`: "No rule to make target 'modules'". Se descargó el árbol `linux-msft-wsl-6.18.54.1`
Prueba de carrera que ignoraba el retorno de `pwrite()`	`-Werror=unused-result` detuvo la compilación	Ahora se cuentan las escrituras aceptadas

También verifiqué contra el código y el historial de Linux las sugerencias comunes que advierte el enunciado, y las descarté:

- Hallar `sys_call_table` con `kallsyms_lookup_name()`: no está exportado desde 5.7 (commit 0bd476e6c671).
- Quitar CR0.WP con `write_cr0()`: el bit está fijado desde 5.3 (commit 8dbec27a242c). Además, sin control de concurrencia, desprotege la memoria para todas las CPU.
- Registrar un LSM desde el módulo con `security_add_hooks()`: es `__init` (`security/security.c:629`).

- El gancho `file_mmap` con 6 argumentos: se dividió en `mmap_addr` y `mmap_file` en 3.5 (commit e5467859f7f7). Hoy es `mmap_file(file, reqprot, prot, flags)` (`include/linux/lsm_hook_defs.h:202`).

Validación manual

1 Descargué el árbol del kernel en ejecución (`linux-msft-wsl-6.18.54.1`) porque los headers de la distribución estaban incompletos. Lo configuré con `/proc/config.gz` y lo compilé con el mismo compilador (gcc 13) para obtener `Module.symvers` (`CONFIG_MODVERSIONS=y`).
2 Busqué con `grep` en ese árbol cada símbolo, firma y bandera del módulo. Comprobé las versiones de cada cambio con el historial de Linux.
3 Compilé el módulo con Kbuild y sin advertencias contra Linux 6.18, y también contra 5.15.224 y 5.10.273 (`x86_64_defconfig`).
4 Cargué y descargué el módulo con `insmod`/`rmmod`, revisé `dmesg` y ejecuté los ataques con y sin el módulo.

Arquitectura

```
Ring 3   mmap(fd, PROT_WRITE, MAP_SHARED)       mprotect(addr, PROT_WRITE)
------------------------------ syscall -------------------------------------
Ring 0   security_mmap_file()   <- kretprobe ->  security_file_mprotect()
             mmap_guard: ¿inode protegido y (escritura compartida o exec)?
             sí -> -EACCES      no -> sigue normal (una comparación)
         do_mmap() -> VMA -> #PF -> filemap_fault() -> PTE   (sin cambios)
```

Las kretprobes se ponen en la entrada a los ganchos LSM, por donde pasa toda creación de mapeos y todo cambio de permisos. Al cargarse, el módulo resuelve el inode protegido. En cada llamada, el manejador de entrada compara ese inode. Si la política niega la petición, el manejador de retorno cambia el resultado a `-EACCES` antes de que exista la VMA. El camino de fallos de página no cambia.

Opción	Latencia de mmap	Page faults	Profundidad de inspección	Cobertura
`LD_PRELOAD`	Ninguna	Sin cambio	Argumentos en libc	Nula: se evita con una syscall directa o un binario estático
Hook en `sys_call_table`	Baja	Sin cambio	Argumentos crudos de usuario (TOCTOU)	Parcial; inviable en 6.x
LSM compilado o BPF LSM	Mínima	Sin cambio	`struct file`, `prot` y `flags` validados	Total al crear y al cambiar permisos; requiere recompilar o `lsm=bpf`
kretprobe en ganchos LSM (elegida)	+260 ns por `mmap()` (medido)	Sin cambio (medido)	Igual a LSM	Igual a LSM; depende de símbolos sondeables
Instrumentar `handle_mm_fault`	Ninguna	Alta: cada fallo	Dirección y tipo de acceso por página	Solo el primer acceso a cada página

Parte III: implementación y resultados

Código

- `kernel/mmap_guard.c`: el módulo. Tiene la política en `verdict()`, las dos kretprobes y el registro con contadores en `/sys/module/mmap_guard/parameters/{allowed,denied}`.
- `kernel/Makefile`: compila el módulo con Kbuild.
- `cat_memory.c`: el comando `m_fmap`.
- Pruebas: `tests/run_mmap_guard_tests.sh` y `tests/mmap_attacks.c`.
- Medición: `tests/mmap_overhead.sh` y `tests/mmap_bench.c`.
- Demostración: `kernel/demo.sh`.

El módulo compila sin advertencias contra Linux 6.18, 5.15 y 5.10, y se ejecutó en 6.18.

Por qué en el kernel

Después del primer fallo, un acceso por puntero no ejecuta ninguna instrucción del kernel. Una biblioteca en espacio de usuario solo puede envolver `mmap()`, y cualquier binario estático o `syscall(SYS_mmap)` la evita. Los ganchos LSM son el único punto obligatorio: crear la VMA o ampliar sus permisos. Negar ahí impide que se instale una PTE escribible, así que no hace falta inspeccionar cada escritura, algo imposible a la velocidad del hardware.

Pruebas

`sh tests/run_mmap_guard_tests.sh` pasó las 18 comprobaciones en Ubuntu 26.04 sobre WSL2, con kernel 6.18.54.1. Sin el módulo, el mismo usuario sí puede escribir el archivo por mmap: ese es el grupo de control. Con el módulo:

- Se permiten la lectura y la copia privada.
- Se rechazan con `EACCES` la escritura compartida, `MAP_SHARED_VALIDATE`, la ejecución y las ampliaciones con `mprotect`.
- En la carrera Dirty COW, el kernel aceptó 153 736 escrituras por `/proc/self/mem`. Todas quedaron en la copia privada y el archivo no cambió.
- `dmesg` registra cada decisión con pid, uid y programa.

strace muestra que el rechazo viene del kernel, en la frontera de la syscall:

```
mmap(NULL, 29, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = -1 EACCES (Permission denied)
```

Sobrecosto

`sh tests/mmap_overhead.sh`: 42 016 llamadas a `mmap()` por corrida (la mitad sobre el archivo protegido), mediana de 3 corridas.

Medida	Herramienta	Sin módulo	Con módulo
Tiempo en kernel por `mmap()` (`vm_mmap_pgoff`)	ftrace function_graph	2640 ns	2899 ns (+10 %)
Tiempo por fallo de página (`handle_mm_fault`)	ftrace function_graph	976 ns	975 ns
Fallos de página atendidos	ftrace	42 070	42 070
`mmap()` por llamada	strace -c	31 µs	31 µs
Ciclo `mmap` + acceso + `munmap`, archivo protegido	`clock_gettime`, mediana de 5	8560 ns	8445 ns

La sonda agrega unos 260 ns por `mmap()`. Ese tiempo incluye la entrada a la kretprobe, la comparación del inode y, para el archivo protegido, la decisión y el registro. Es un 3 % del ciclo completo, menos que la variación entre corridas (7771 a 8883 ns). El tiempo y la cantidad de fallos de página no cambian, porque el módulo no toca `handle_mm_fault`. Con strace, el costo de ptrace (unos 30 µs por llamada) oculta la diferencia, por eso ftrace es la medida de referencia. ftrace también suma su propio costo, pero las dos corridas lo pagan por igual.

Limitaciones y trabajo futuro

Vectores aún descubiertos:

- `write()`, `pwrite()` y `ftruncate()` siguen dependiendo de los permisos POSIX. Se cubrirían con un gancho en `security_file_permission`.
- Los mapeos creados antes de cargar el módulo siguen activos.
- Root puede descargar el módulo o desactivar todos los kprobes (`/sys/kernel/debug/kprobes/enabled`).
- Un montaje overlayfs puede presentar otro inode y no coincidir con el protegido.
- Se protege un solo archivo, por inode.

Mitigaciones de hardware e hipervisor:

- Fijar la política desde el arranque, con BPF LSM o un LSM compilado, lockdown y módulos firmados. El hipervisor puede además dejar la memoria del kernel en solo lectura con EPT, como hace HVCI.
- Usar IOMMU contra el acceso por DMA, Intel TME o AMD SME contra la lectura física de la RAM, y SEV-SNP o TDX contra un hipervisor comprometido.